

Отчёт по практической работе №1

Группа ЕТ-442 Замятин Андрей Александрович

1 Задание

Требовалось написать на языке Python 3 программу которая создает и обучает нейронную сеть для распознавания рукописных цифр с использованием метода градиентного спуска и базы данных MNIST. Работа выполнена по указаниям к практической работе 1.

2 Исходные данные

Сеть содержит три слоя. Входной слой принимает изображения размером 28 на 28 пикселей. Это дает 784 входа. Скрытый слой содержит 30 нейронов. Выходной слой содержит 10 нейронов по числу цифр от 0 до 9. Используются язык Python версии 3.12.3 и библиотека NumPy версии 1.26.4. Файл данных mnist.pkl.gz размером 16 мегабайт заранее загружен.

3 Состав программы

Программа состоит из двух модулей. Модуль network.py описывает класс Network и функции активации. Модуль mnist_loader.py загружает файл mnist.pkl.gz и приводит наборы к виду удобному для обучения. Набор обучения содержит 50000 пар входного изображения и ожидаемого вектора. Вектор ожидаемого ответа имеет длину 10 и содержит единицу в позиции верной цифры. Набор проверки содержит 10000 пар. Набор тестирования содержит 10000 пар. Такое разделение соответствует структуре базы MNIST.

4 Алгоритм

Для активации использована сигмоидальная функция. Она вычисляется через экспоненту из NumPy в векторизованной форме и применяется поэлементно. Ее производная выражена через значение самой функции. Это избавляет от повторного вычисления экспоненты.

Прямое распространение последовательно вычисляет потенциалы слоев как произведение матрицы весов на сигналы предыдущего слоя с добавлением смещения. Затем к потенциалам применяется активация. Одного вызова метода прямого распространения достаточно для получения ответа сети.

Обучение выполнено методом стохастического градиентного спуска. В начале каждой эпохи обучающая выборка перемешивается случайным образом. Затем она делится на малые подвыборки размером 10. Для каждой подвыборки вызывается один шаг обновления весов и смещений.

Градиенты для каждой пары из подвыборки вычисляются методом обратного распространения. Сначала выполняется прямой проход с запоминанием потенциалов и активаций всех слоев. Ошибка выходного слоя находится как разность выхода сети и ожидаемого вектора умноженная на производную активации. Ошибка скрытых слоев находится через транспонированные матрицы весов со сдвигом по слоям назад. Затем для каждого слоя вычисляются градиенты по смещениям и по весам. Градиенты всех пар подвыборки суммируются.

Веса и смещения обновляются вычитанием среднего градиента умноженного на скорость обучения 3.0. После завершения эпохи вызывается оценка точности на тестовом наборе. Ответом сети считается номер выходного нейрона с наибольшей активацией. Подсчитывается число верных ответов из 10000. В указаниях приведен ориентир роста точности до значений около 9300 верных из 10000 к средним эпохам при 30 эпохах обучения.

5 Запуск в используемой системе

В указаниях описан запуск через среду IDLE в Windows с ручной установкой рабочего каталога. Фактическая работа выполнена в Linux Ubuntu 24.04.5 LTS на архитектуре x86_64. Поэтому запуск выполнен из терминала интерпретатором Python напрямую без IDLE.

Текущий каталог предварительно установлен в каталог с файлами программы и базой данных. Это важно так как загрузчик открывает файл mnist.pkl.gz по относительному имени. Проверка показала 50000 пар в обучении и по 10000 пар в проверке и тестировании. Инициализация сети с топологией 784 30 10 прошла без ошибок.

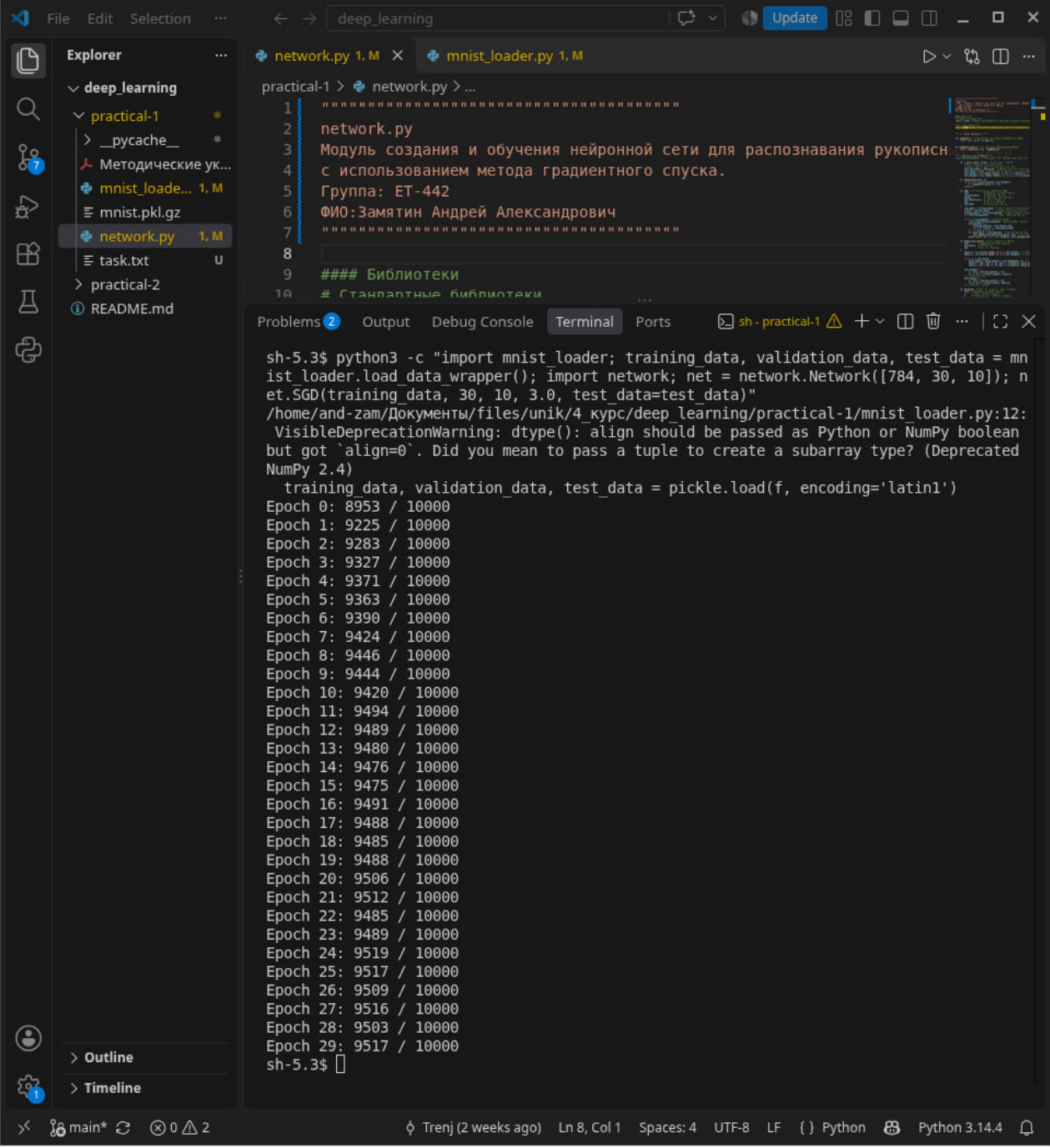
Для запуска обучения использованы последовательные команды интерпретатора.

```
import mnist_loader
training_data, validation_data, test_data = mnist_loader.load_data_wrapper()
import network
net = network.Network([784, 30, 10])
net.SGD(training_data, 30, 10, 3.0, test_data = test_data)
```

Та же команда записана в файле task.txt для удобного копирования. Обучение занимает несколько минут. В ходе обучения на экран выводится строка для каждой эпохи с номером эпохи и числом верных ответов из 10000. Полная команда с однострочным вызовом дает тот же результат но в отчете она разбита на строки чтобы избежать разделителей.

6 Результат

Обучение выполнено за 30 эпох. Вывод программы полностью сохранен на скриншоте ниже. На нулевой эпохе получено 8953 верных ответа из 10000. Это начальный уровень после первой эпохи. Далее точность росла. На последней эпохе с номером 29 получено 9517 верных из 10000. Наилучший результат 9519 верных из 10000 зафиксирован на эпохе с номером 24. Такой ход подтверждает корректность загрузки данных и работы методов обучения и оценки.



The screenshot shows a VS Code editor with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The file explorer shows a project named 'deep_learning' with a subdirectory 'practical-1' containing files like 'mnist_loader.py', 'network.py', 'task.txt', and 'README.md'. The code editor shows the 'network.py' file with comments in Russian and a header with the author's name. The terminal shows the execution of a Python script that loads data and trains a neural network for 30 epochs. The output displays the accuracy for each epoch, starting at 8953 and peaking at 9519 in epoch 24, before ending at 9517 in epoch 29.

```
practical-1 > network.py > ...
1 #####
2 network.py
3 Модуль создания и обучения нейронной сети для распознавания рукописн
4 с использованием метода градиентного спуска.
5 Группа: ЕТ-442
6 ФИО:Замятин Андрей Александрович
7 #####
8
9 #### Библиотеки
10 # Стандартные библиотеки
...

sh-5.3$ python3 -c "import mnist_loader; training_data, validation_data, test_data = mn
ist_loader.load_data_wrapper(); import network; net = network.Network([784, 30, 10]); n
et.SGD(training_data, 30, 10, 3.0, test_data=test_data)"
/home/and-zam/Документы/files/unik/4_kypc/deep_learning/practical-1/mnist_loader.py:12:
VisibleDeprecationWarning: dtype(): align should be passed as Python or NumPy boolean
but got 'align=0'. Did you mean to pass a tuple to create a subarray type? (Deprecated
NumPy 2.4)
training_data, validation_data, test_data = pickle.load(f, encoding='latin1')
Epoch 0: 8953 / 10000
Epoch 1: 9225 / 10000
Epoch 2: 9283 / 10000
Epoch 3: 9327 / 10000
Epoch 4: 9371 / 10000
Epoch 5: 9363 / 10000
Epoch 6: 9390 / 10000
Epoch 7: 9424 / 10000
Epoch 8: 9446 / 10000
Epoch 9: 9444 / 10000
Epoch 10: 9420 / 10000
Epoch 11: 9494 / 10000
Epoch 12: 9489 / 10000
Epoch 13: 9480 / 10000
Epoch 14: 9476 / 10000
Epoch 15: 9475 / 10000
Epoch 16: 9491 / 10000
Epoch 17: 9488 / 10000
Epoch 18: 9485 / 10000
Epoch 19: 9488 / 10000
Epoch 20: 9506 / 10000
Epoch 21: 9512 / 10000
Epoch 22: 9485 / 10000
Epoch 23: 9489 / 10000
Epoch 24: 9519 / 10000
Epoch 25: 9517 / 10000
Epoch 26: 9509 / 10000
Epoch 27: 9516 / 10000
Epoch 28: 9503 / 10000
Epoch 29: 9517 / 10000
sh-5.3$
```

Скриншот терминала с выводом обучения сети за 30 эпох

7 Вывод

Программа собрана в полном соответствии с указаниями. Все методы класса Network реализованы. Загрузка данных MNIST выполнена корректно с приведением размерностей к виду 784 на 1 для входов и 10 на 1 для ответов. Топология сети 784 30 10 соответствует заданию с тремя слоями и входом 28 на 28. Обучение сходится. Точность на тестовом наборе растет по эпохам что видно по скриншоту.

Исходный код

Актуальная версия программы для данной практики хранится по адресу https://github.com/Trenj/deep_learning/tree/main/practical-1